

## Setter Injection vs Constructor Injection and how to set the reference Type

===== Que - Which is Optional and Which is Mandatory? 1. What happens if we don't inject any value at all? In Java, if we don't set any value and just create the object, calling the reference variable will still print the object — but the values will not be set or injected. As a result, we get the default values. Student student = new Student(); System.out.println(student); Output: Student [name=null, id=0, address=null] The same thing happens if we don't provide setter injection using the tag in Spring. It will behave the same way. ApplicationContext context = new ClassPathXmlApplicationContext("com/config/application-context.xml"); Student bean = context.getBean("studentclass", Student.class); System.out.println(bean); Output: Student [name=null, id=0, address=null] -----

What happens if we remove all setters from the bean class? Now let's comment out all the setters from our bean class and try creating the object again. In the traditional way, we still get the same default values. Student [name=null, id=0, address=null] We get the same result even with the IOC container — since no property injection was configured, Spring simply creates the object of our class, just like the traditional approach. -----

----- What happens if we don't write any constructor? Now let's not write any constructor in our bean class — we'll only provide setters, getters, and toString(). This time, we configure setter injection using the tag in Spring. Since no constructor is written, the compiler provides a default constructor automatically, and then the values are injected through the setters. Traditional way: Student id setter called Student address setter called Student name setter called Student [name=prashant, id=101, address=hyd] Spring way: Student name setter called Student id setter called Student address setter called Student [name=prashant, id=101, address=hyd] This shows that setter injection works even without writing an explicit constructor — which means optional is setter injection. -----

----- What happens if we remove the constructor but configure constructor injection? Now suppose we remove the constructor from our class, but in the configuration file we still write constructor injection (). Even though our class has the default constructor (given by the compiler), it will not execute with the Spring approach. Spring looks for a matching parameterized constructor for constructor injection, and since it isn't there, the default constructor is not used automatically. This proves that constructor injection is mandatory — the matching constructor must exist in the class for it to work. -----

----- How does injection work when one class depends on another class? Let's check what happens when our class depends on another class, and how injection takes place. Without creating the Resume class object, if we try to create the Student class object using constructor injection, will it compile successfully? No — because the parameters won't match. In the traditional way, we get a compile-time error. In the Spring way, we get an exception. But if we do the same thing using setter injection, without creating the Resume object, we get: Traditional way: Student class default constructor Student id setter called Student address setter called Student name setter called Student [name=prashant, id=101, address=hyd, resume=null] Spring way: Student class default constructor Student name setter called Student id setter called Student address setter called Student [name=prashant, id=101, address=hyd, resume=null] -----

Now let's see the code for how to inject the dependency of one class into another. Student.java -----

```
----- package core.BasicProgram; public class Student {  
private String name; private int id; private String address; private Resume resume; public  
Student() { System.out.println("Student class default constructor"); } public Student(String name,  
int id, String address, Resume resume) { super(); System.out.println("Parameterized constructor  
"); this.name = name; this.id = id; this.address = address; this.resume = resume; } public String  
getName() { return name; } public void setName(String name) { System.out.println("Student  
name setter called"); this.name = name; } public int getId() { return id; } public void setId(int id) {  
System.out.println("Student id setter called"); this.id = id; } public String getAddress() { return  
address; } public void setAddress(String address) { System.out.println("Student address setter  
called"); this.address = address; } public Resume getResume() { return resume; } public void  
setResume(Resume resume) { this.resume = resume; } @Override public String toString() {  
return "Student [name=" + name + ", id=" + id + ", address=" + address + ", resume=" + resume
```

```

+ "]; } } ----- Resume.java -----
----- package core.BasicProgram; public class
Resume { private int experience; private String domain; public Resume() {
System.out.println("Resume class default constructor"); } public Resume(int experience, String
domain) { super(); this.experience = experience; this.domain = domain; } public int
getExperience() { return experience; } public void setExperience(int experience) {
this.experience = experience; } public String getDomain() { return domain; } public void
setDomain(String domain) { this.domain = domain; } @Override public String toString() { return
"Resume [experience=" + experience + ", domain=" + domain + "]; } } -----
----- application-context.xml -----
----- App.java -----
----- public class App { public static void
main(String[] args) { System.out.println("Hello World!"); System.out.println("Traditional way ");
Resume resume = new Resume(1, "java"); // resume.setDomain("java"); //
resume.setExperience(1); // Student student = new Student(); Student student = new
Student("prashant", 101, "hyd", resume); // student.setId(101); // student.setAddress("hyd"); //
student.setName("prashant"); // student.setResume(resume); System.out.println(student);
System.out.println("====="); System.out.println("By spring way");
ApplicationContext context = new ClassPathXmlApplicationContext("com/config/application-
context.xml"); Student bean = context.getBean("studentclass", Student.class);
System.out.println(bean); } }

```